

Session 13 (AIML) -Assignment Question

- **Name of Student :** Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic , Narhe
- **Branch Name:** AIML

Q1.

PROGRAM:

```
#Q1
import numpy as np

# a) 1D Array
arr1 = np.array([10, 20, 30, 40, 50])

# b) 2D Array (3x3)
arr2 = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

print("1D Array:")
print(arr1)
print("Shape:", arr1.shape)
print("Data Type:", arr1.dtype)

print("\n2D Array:")
print(arr2)
print("Shape:", arr2.shape)
print("Data Type:", arr2.dtype)
```

OUTPUT:

```
1D Array:
[10 20 30 40 50]
Shape: (5,)
Data Type: int64
```

```
2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)
Data Type: int64
```

Q2.

PROGRAM:

```
#Q2

import numpy as np

a = np.zeros(8)
b = np.ones((4, 4))
c = np.zeros((3, 3))

print("1D Array of 8 Zeros:")
print(a)

print("\n4x4 Array of Ones:")
print(b)

print("\n3x3 Matrix of Zeros:")
print(c)
```

OUTPUT:

```
1D Array of 8 Zeros:
[0. 0. 0. 0. 0. 0. 0. 0.]
```

```
4x4 Array of Ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

```
3x3 Matrix of Zeros:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Q3.

PROGRAM:

```
#Q3

import numpy as np

a = np.arange(0, 21, 1)
b = np.arange(10, 51, 2)
c = np.arange(5, 101, 5)

print("Numbers from 0 to 20:")
print(a)

print("\nEven Numbers from 10 to 50:")
print(b)

print("\nNumbers from 5 to 100 (Step 5):")
print(c)
```

OUTPUT:

Numbers from 0 to 20:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

Even Numbers from 10 to 50:

```
[10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50]
```

Numbers from 5 to 100 (Step 5):

```
[ 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90
 95 100]
```

Q4.

PROGRAM:

```
#Q4

import numpy as np

a = np.linspace(0, 5, 10)
b = np.linspace(-10, 10, 15)

print("Between 0 and 5:")
print(a)
print("Length:", len(a))

print("\nBetween -10 and 10:")
print(b)
print("Length:", len(b))
```

OUTPUT:

```
Between 0 and 5:
[0.          0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.         ]
Length: 10

Between -10 and 10:
[-10.         -8.57142857 -7.14285714 -5.71428571 -4.28571429
 -2.85714286 -1.42857143  0.          1.42857143  2.85714286
 4.28571429  5.71428571  7.14285714  8.57142857 10.         ]
Length: 15
```

Q5.

PROGRAM:

```
#Q5

import numpy as np

a = np.random.rand(10)

b = np.random.randn(3, 3)

c = np.random.randint(10, 51, (4, 5))

print("Random Numbers (0 to 1):")
print(a)

print("\nRandom Standard Normal Matrix:")
print(b)

print("\nRandom Integer Matrix (10-50):")
print(c)
```

OUTPUT:

```
Random Numbers (0 to 1):
[0.85226273 0.05382051 0.18549339 0.50144589 0.09307432 0.60015869
 0.21818753 0.41010402 0.94687284 0.79792422]
```

```
Random Standard Normal Matrix:
[[ 0.84076744 -0.82090301  0.16908745]
 [-0.53731878  0.12627179 -0.16930575]
 [-0.10801331  0.2542437  0.09723177]]
```

```
Random Integer Matrix (10-50):
[[25 31 10 35 10]
 [19 32 50 24 41]
 [31 16 47 36 10]
 [38 27 16 27 39]]
```

Q6.

PROGRAM:

```
#Q6

import numpy as np

v1 = np.array([2, 4, 6, 8])
v2 = np.array([1, 3, 5, 7])

print("Addition:")
print(v1 + v2)

print("\nSubtraction:")
print(v1 - v2)

print("\nElement-wise Multiplication:")
print(v1 * v2)

print("\nDot Product:")
print(np.dot(v1, v2))
```

OUTPUT:

```
Addition:
[ 3  7 11 15]

Subtraction:
[1 1 1 1]

Element-wise Multiplication:
[ 2 12 30 56]

Dot Product:
100
```

Q7.

PROGRAM:

```
#Q7

import numpy as np

A = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

B = np.array([[9,8,7],
              [6,5,4],
              [3,2,1]])

print("Matrix Addition:")
print(A + B)

print("\nMatrix Multiplication:")
print(A @ B)

print("\nElement-wise Multiplication:")
print(A * B)
```

OUTPUT:

```
Matrix Addition:
[[10 10 10]
 [10 10 10]
 [10 10 10]]

Matrix Multiplication:
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]

Element-wise Multiplication:
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]
```

Q8.

PROGRAM:

```
#Q8

import numpy as np

A = np.random.randint(1, 101, (4, 4))

print("Matrix:")
print(A)

print("\nShape:", A.shape)
print("Dimension:", A.ndim)
print("Size:", A.size)
print("Data Type:", A.dtype)
print("Minimum Value:", A.min())
print("Maximum Value:", A.max())
```

OUTPUT:

```
Matrix:
[[36 73  6 19]
 [85 36 15 64]
 [15 76 11 11]
 [94 46 73 77]]

Shape: (4, 4)
Dimension: 2
Size: 16
Data Type: int32
Minimum Value: 6
Maximum Value: 94
```

Q9.

PROGRAM:

```
#Q9

import numpy as np

arr = np.random.randint(1, 51, 20)

matrix = arr.reshape(4, 5)

print("Matrix:")
print(matrix)

print("\nSum:", np.sum(matrix))
print("Mean:", np.mean(matrix))
print("Standard Deviation:", np.std(matrix))

print("\nMaximum Value in Each Row:")
print(np.max(matrix, axis=1))
```

OUTPUT:

```
Matrix:
[[28 48  9 36 36]
 [ 3 19 48 45 28]
 [34  8 37 29 21]
 [42 28  9 34 15]]

Sum: 557
Mean: 27.85
Standard Deviation: 13.439773063560262

Maximum Value in Each Row:
[48 48 37 42]
```

Q10.

PROGRAM:

```
#Q10

import numpy as np

try:
    n = int(input("How many random numbers do you want to generate? "))

    if n <= 0:
        print("Please enter a positive number.")

    else:
        arr = np.random.randint(10, 101, n)

        print("\nGenerated Array:")
        print(arr)

        print("\nMean:", np.mean(arr))
        print("Median:", np.median(arr))
        print("Standard Deviation:", np.std(arr))
        print("Minimum:", np.min(arr))
        print("Maximum:", np.max(arr))

        if n % 2 == 0:
            matrix = arr.reshape(2, n // 2)

            print("\nReshaped 2D Array:")
            print(matrix)

            print("\nRow-wise Sum:")
            print(np.sum(matrix, axis=1))
        else:
            print("\nCannot reshape into a 2D array because the number of elements is odd.")

except ValueError:
    print("Invalid input! Please enter an integer.")
```

OUTPUT:

How many random numbers do you want to generate? 25

Generated Array:

```
[ 81 11 42 100 96 39 22 68 40 53 84 25 23 68 78 97 35 71
 35 38 15 40 48 75 77]
```

Mean: 54.44

Median: 48.0

Standard Deviation: 26.496913027747215

Minimum: 11

Maximum: 100

Cannot reshape into a 2D array because the number of elements is odd.